

# AI-first Software Engineering Program – Case Study

## TrustDesk: AI Support Operations Agent

Author : **Airtribe**

### BACKGROUND

Modern support teams handle thousands of repetitive but high-stakes customer conversations. A good support system must answer quickly, cite the right policy, understand customer and order context, route edge cases to humans, and avoid taking risky actions without approval.

Traditional chatbots usually fail because they either hallucinate answers, ignore company policy, or cannot safely act on behalf of support teams. TrustDesk is an AI-first product where the AI is not just a text generator, it must retrieve policy context, classify tickets, decide whether it has enough evidence, suggest or execute allowed tools, and leave a complete audit trail.

### OBJECTIVE

Build a reliable AI support operations platform that can:

- Ingest company support documents and retrieve relevant policy context.
- Triage incoming support tickets by intent, priority, and escalation need.
- Generate grounded draft replies with citations from the knowledge base.
- Suggest safe operational actions - replacement, refund review, coupon creation, or escalation.
- Require human approval before executing sensitive actions.
- Defend against prompt injection, unsupported policy claims, PII leakage, and excessive agency.
- Track evaluations and the minimum trace data needed to debug AI decisions.

## KEY FEATURES

### Load the Provided Data

- Load customers, orders, tickets, tool catalog, eval cases, and knowledge-base documents.
- Preserve document IDs (e.g., KB-REFUND-001) — evals depend on them.
- Evaluate return/warranty windows against each ticket's created\_at, never the current date.

### Ticket APIs and Simple Frontend

- APIs to list tickets, fetch a ticket, and view linked customer/order context.
- A simple support-agent frontend (vibe-coded is fine) to open a ticket, trigger triage, generate a draft, view citations, review one sensitive action, and see eval results.

### Knowledge Retrieval & Cited Draft Replies

- Search knowledge-base docs for policy context relevant to a ticket.
- Generate a draft using ticket + customer/order context + retrieved documents, with citation IDs.
- Refuse or escalate when the answer isn't supported by retrieved policy.

### AI Triage

- Classify into a category (shipping, refund, warranty, billing, account\_security, general).
- Assign priority (low, medium, high, urgent) and decide on escalation.

### One Approval-Gated Tool Action

- Implement start\_refund\_review or create\_replacement\_order.
- AI recommends; a human must approve before execution.
- Enforce an idempotency key so retries can't create duplicate actions.

### Guardrails for Adversarial Cases

- Treat customer messages and retrieved documents as untrusted input.
- Correctly handle: identity-check bypass requests, hidden-coupon prompt injection, and requests to reveal hidden prompts/secrets.
- KB-ADVERSARIAL-001 is an intentionally unsafe document — must not be followed as an instruction.

### Minimal Traces & Evaluation Runner

- Store a trace per AI run: ticket ID, run type, retrieved doc IDs, tool actions, guardrail result, final status.
- One command/endpoint/flow to run data/eval\_cases.jsonl and produce a summary report (triage accuracy, citation coverage, unsafe-action blocking, escalation behavior).

## TECHNICAL REQUIREMENTS

- Expose core functionality as RESTful APIs (or clearly documented equivalent).
- Use a reliable persistent store for tickets, customers, orders, documents, drafts, tool calls, approvals, and traces.
- Implement simple authentication (demo token or login flow); full RBAC is Good-to-Have.
- Implement a retrieval layer: full-text search, vector DB, hybrid, or documented local substitute.
- Keep the AI provider behind an adapter so it can be mocked in tests.
- Use idempotency for the approval-gated tool action.
- Provide automated tests for critical flows and guardrails.
- Avoid blocking unrelated requests during long-running AI/eval operations.
- Provide one command/endpoint to run eval cases and produce a summary.
- No dependency on any specific language, framework, or hosted AI provider..

## ASSESSMENT CRITERIA

- Must Have Workflow - 60%: Data loading, required APIs, simple frontend, triage, cited drafts, one approval-gated action with idempotency, minimal traces, and evals, all working.
- AI Quality & Guardrails - 25%: Answers grounded in policy, cited correctly, escalated when uncertain, safe on adversarial eval cases.
- Engineering Quality, Documentation & Demo - 15%: Maintainable code, clear setup, documented design decisions, demo clearly shows the core workflow.

*Good-to-Have and Stretch work can strengthen the project but doesn't compensate for missing Must-Have functionality.*

## DELIVERABLES

1. The final, functional product.
2. README file covering setup instructions, API documentation, architecture, design decisions, evaluation flow, and known limitations.
3. Public link of the GitHub repository.
4. Explainer video demonstrating the project workflow, including ticket triage, cited draft generation, approval-gated action, guardrail handling, and evaluation results.
5. Evaluation summary showing triage accuracy, citation coverage, unsafe-action blocking, and escalation behavior.

## RESOURCES

**GitHub Repository:**

**<https://github.com/airtribe-projects/trustdesk-capstone>**